CHAPTER 04

EC2 & Auto Scaling

Instance Types · ASG · ALB · EBS · Spot · Savings Plans

04-00 Chapter Overview

Topic	AWS Service	ShopFlow Use-Case
EC2 Instance Types	EC2: t3, m6g, c6g, r6g families	Graviton saves 40% vs x86 for Java
AMIs & Launch Templates	EC2 Image Builder, Launch Template v2	Golden AMI quarterly — 90s boot time
Auto Scaling Groups	ASG, Target Tracking, Step, Scheduled	Scale 2 to 40 on Black Friday events
Application Load Balancer	ALB, NLB, Target Groups, ALB Rules	Path routing to 5 microservices
EBS Block Storage	gp3, io2, st1, sc1 volume types	All volumes gp3 encrypted by default
Spot Instances	Spot Fleet, Interruption Handler	Image batch processing at 90% discount
Placement Groups	Cluster, Partition, Spread	Fault-isolated critical service nodes
Systems Manager	Patch Manager, Session Manager	No-SSH OS patching, ECS Exec debugging
CloudWatch Agent	Custom metrics, log shipping	Memory + disk for ASG scaling decisions
Lifecycle Hooks	LAUNCHING and TERMINATING hooks	Graceful drain before scale-in events
Savings Plans & RI	Compute SP, EC2 SP, Reserved Instances	Commit to baseline — $563/month saved
Observability & Alarms	CloudWatch metrics, Log Insights	Full-stack EC2 monitoring dashboards

04-01 EC2 Instance Types — Graviton vs x86

AWS offers 700+ EC2 instance types across 5 categories. Choosing incorrectly wastes 30-70% of compute spend. For ShopFlow's Java 21 Spring Boot microservices, Graviton3 (ARM64) instances deliver 40-50% better price/performance than equivalent Intel/AMD instances, with full Java and Docker compatibility.

Instance Family Reference Guide

Family	Category	vCPU:RAM Ratio	Example Instance	On-Demand/hr	ShopFlow Role
t3/t4g	Burstable CPU	1:2	t3.medium 2vCPU/4GB	$0.042	Dev environments, bastion
m6g/m7g	General (Graviton)	1:4	m6g.xlarge 4vCPU/16GB	$0.154	ECS compute baseline tier
c6g/c7g	Compute (Graviton)	1:2	c6g.xlarge 4vCPU/8GB	$0.136	API servers, product-svc
r6g/r7g	Memory (Graviton)	1:8	r6g.xlarge 4vCPU/32GB	$0.252	Redis, OpenSearch data nodes
i3en/im4gn	Storage NVMe	1:8	im4gn.xlarge 4vCPU/32GB	$0.434	High-IOPS DB (not ShopFlow)
p3/g4dn	GPU Compute	varies	g4dn.xlarge 4vCPU+T4 GPU	$0.526	ML training (future phase)

Graviton vs x86 — ShopFlow Benchmark Results

Benchmark Metric	x86 c5.xlarge ($0.170/hr)	Graviton2 c6g.xlarge ($0.136/hr)	Graviton3 c7g.xlarge ($0.145/hr)
Monthly cost (730 hours)	$124/month	$99/month (20% cheaper)	$106/month (15% cheaper)

Benchmark Metric	x86 c5.xlarge ($0.170/hr)	Graviton2 c6g.xlarge ($0.136/hr)	Graviton3 c7g.xlarge ($0.145/hr)
Java Spring Boot TPS	Baseline (100%)	135-140% of baseline	+145-150% of baseline
ShopFlow product-svc p99	82ms latency	51ms (38% improvement)	47ms (43% improvement)
GC pause time (G1GC)	45ms avg pause	28ms avg pause	22ms avg pause
Cost per 1M API requests	$2.48	$1.49 (-40%)	$1.27 (-49%)
Docker image size	x86 layers only	Must rebuild for arm64 layers	Same as c6g

Multi-Arch Docker Build for Graviton

```
# Build multi-arch image (runs on both x86 and ARM64)
docker buildx create --name multiarch --use
docker buildx build \
--platform linux/amd64,linux/arm64 \
--tag 123456789012.dkr.ecr.us-east-1.amazonaws.com/shopflow/product-svc:v1.2.3 \
--push \
./services/product-svc/
# ECS task definition: just change the instance type — same image works
"cpu": "1024", "memory": "2048",
"runtimePlatform": {
"cpuArchitecture": "ARM64", <- change x86_64 to ARM64
"operatingSystemFamily": "LINUX"
}
```

■ Switch incrementally: deploy one ECS service at a time to Graviton. Monitor p99 latency and error rates for 48 hours. If stable, switch the next service. ShopFlow migrated all 5 services to c6g in 2 weeks.

04-02 **AMIs & Launch Templates — Golden AMI Strategy**

A Golden AMI is a pre-baked machine image with all dependencies installed, security-hardened, and compliance-validated. ShopFlow builds a new Golden AMI quarterly using EC2 Image Builder, enabling new instances to reach ready state in under 90 seconds (vs 10-20 min for User-Data bootstrap).

Golden AMI vs User-Data Bootstrap Comparison

Aspect	User-Data Script	Golden AMI (Pre-Baked)
Boot time	10-20 minutes (yum install at runtime)	60-90 seconds (just start app)
Consistency	Varies (external package repo state)	100% reproducible from snapshot
External dependency risk	Package repo down = broken instance	None — all packages baked in
CVE scanning	Not possible at boot	Image Builder scans AMI before publish
CIS hardening	Complex, error-prone at runtime	Part of Image Builder recipe
ShopFlow production standard	Dev only (quick iteration)	YES — all production EC2 instances

EC2 Image Builder Pipeline — CDK

```
// Image Builder recipe: AL2023 → patch → Java21 → agents → CIS → test
const recipe = new imagebuilder.CfnImageRecipe(this,"GoldenAMI",{
name: "shopflow-golden-ami",
version: "1.0.0",
parentImage: `arn:aws:imagebuilder:${region}:aws:`+
"image/amazon-linux-2023-arm64/x.x.x",
components: [
{componentArn: "arn:aws:imagebuilder:::component/aws-cli-version-2-linux/x.x.x"},
{componentArn: java21Component.attrArn}, // Install Corretto 21
{componentArn: cwAgentComponent.attrArn}, // CloudWatch Agent 1.3
```

```
{componentArn: ssmAgentComponent.attrArn}, // SSM Agent latest
{componentArn: cisL1Component.attrArn}, // CIS Amazon Linux 2023 L1
{componentArn: validateComponent.attrArn}, // Test: java -version, agent checks
],
blockDeviceMappings: [{deviceName:"/dev/xvda",
ebs:{volumeSize:50,volumeType:"gp3",encrypted:true}}],
});
```

Launch Template — Production Config

```
const lt = new ec2.LaunchTemplate(this,"ShopFlowLT",{
instanceType: ec2.InstanceType.of(ec2.InstanceClass.C6G,ec2.InstanceSize.XLARGE),
machineImage: ec2.MachineImage.genericLinux({
"us-east-1": "ami-shopflow-golden-20241001"}),
securityGroup: sgApp,
role: ec2AppRole,
requireImdsv2: true, // CRITICAL: prevent SSRF credential theft
blockDevices: [{deviceName:"/dev/xvda",
volume: ec2.BlockDeviceVolume.ebs(50,{
volumeType:ec2.EbsDeviceVolumeType.GP3,
encrypted:true, iops:3000, throughput:125})}],
userData: ec2.UserData.forLinux(),
});
```

Launch Template Feature	Status in LT	Status in Launch Config
Multiple versions ($Default/$Latest)	Yes — version history	No — immutable, must replace
Mixed Spot + On-Demand in one ASG	Yes	No
IMDSv2 enforcement (requireImdsv2)	Yes	No
T2/T3 unlimited burst mode	Yes	No
Capacity Reservations	Yes	No
Status as of 2024	Recommended	Deprecated — AWS will retire soon

■ Always set requireImdsv2: true on Launch Templates. IMDSv2 requires a PUT request to obtain a session token before metadata access, blocking SSRF attacks that try to steal EC2 role credentials from 169.254.169.254.

04-03 Auto Scaling Groups — Scaling Policies & Config

Auto Scaling Groups maintain EC2 fleet size between configured min and max bounds, automatically launching or terminating instances based on demand. ShopFlow uses a mixed instance policy (70% On-Demand + 30% Spot) with three simultaneous scaling policies for cost-efficient reliability.

Scaling Policy Comparison

Policy	Trigger Mechanism	Response Time	Precision	ShopFlow Usage
Target Tracking	Metric stays at target X%	1-3 minutes (auto-calibrates)	Self-adjusting, no tuning	product-svc: CPU@60%, 1K req/min/t
Step Scaling	CloudWatch alarm thresholds	1-2 minutes after alarm	High — define exact steps	order-svc: +3 when CPU>80%, +5 >9
Scheduled Scaling	Cron expression (time-based)	Instant at scheduled time	Exact desired capacity	Black Friday 5:30pm: min=10, desired
Predictive Scaling	ML from 14-day CloudWatch history	Pre-scales 10min before spike	Learns weekly patterns	Monday morning 9am rush auto-hand

ASG CDK — Full Production Configuration

```
const asg = new autoscaling.AutoScalingGroup(this,"ProductAsg",{
vpc, launchTemplate: lt,
minCapacity: 2, // always-on, one per AZ
maxCapacity: 20, // Black Friday ceiling (extend to 40 on event day)
desiredCapacity: 4, // normal load: 4 tasks for 1K req/s
vpcSubnets: {subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS},
healthCheck: autoscaling.HealthCheck.elb({
grace: Duration.seconds(90), // wait for Spring Boot JVM warmup
}),
defaultInstanceWarmup: Duration.seconds(60),
cooldown: Duration.seconds(300),
mixedInstancesPolicy: {
instancesDistribution: {
onDemandBaseCapacity: 2, // first 2 always OD
onDemandPercentageAboveBaseCapacity: 70, // 70% OD, 30% Spot
spotAllocationStrategy: "price-capacity-optimized",
},
launchTemplateOverrides: [
{instanceType: new ec2.InstanceType("c6g.xlarge")},
{instanceType: new ec2.InstanceType("c7g.xlarge")},
{instanceType: new ec2.InstanceType("m6g.xlarge")}, // fallback
],
},
updatePolicy: autoscaling.UpdatePolicy.rollingUpdate({
maxBatchSize: 2,
minInstancesInService: 2,
waitOnResourceSignals: false,
}),
});
// Combine multiple scaling policies
asg.scaleOnCpuUtilization("Cpu60",{targetUtilizationPercent:60});
asg.scaleOnRequestCount("Req1K",{targetRequestsPerMinute:1000});
```

■ Set defaultInstanceWarmup to 60s (JVM startup time). Without it, ASG counts new instances in aggregated metrics immediately, causing premature scale-in decisions before new instances are actually ready to serve traffic.

04-04 Application Load Balancer — Layer 7 Routing

The ALB routes based on HTTP path, hostname, headers, and query strings. ShopFlow uses one ALB for all 5 microservices, with path-based rules routing requests to the correct target group. ALB handles HTTPS termination with ACM certificates that auto-renew.

ALB vs NLB — When to Choose Which

Feature	ALB (Layer 7 HTTP)	NLB (Layer 4 TCP/UDP)
Routing intelligence	Path, host, HTTP header, query string, method	IP address + port only — no app context
WebSocket support	Yes — native HTTP Upgrade protocol	Yes — TCP pass-through
gRPC support	Yes — native gRPC routing	TLS pass-through only
Static IP per AZ	Via Global Accelerator only	Yes — assign Elastic IPs directly
Maximum throughput	Millions req/sec (scales automatically)	Millions connections/sec, sub-ms latency
TLS termination	Yes — ACM certificate, auto-renewed	Yes — NLB TLS listener
Cost (hourly + per LCU)	$0.016/hr + $0.008/LCU-hr	$0.0225/hr + $0.006/NLCU-hr
Lambda as target	Yes — ALB → Lambda (no API GW needed)	No
ShopFlow use	All 5 microservices + canary routing	Not currently used

ALB Routing Rules — ShopFlow Config

```
Listener: HTTPS :443 (ACM wildcard *.shopflow.com, TLS 1.2 minimum)
Priority 1: path-pattern /api/v1/products* → product-svc TG (port 8081)
Priority 2: path-pattern /api/v1/orders* → order-svc TG (port 8082)
Priority 3: path-pattern /api/v1/users* → user-svc TG (port 8083)
Priority 4: path-pattern /api/v1/search* → search-svc TG (port 8084)
Priority 5: path-pattern /ws/* → notify-svc TG (port 8085)
Priority 6: http-header X-Beta-User=true → canary-svc TG (10% A/B test)
Default: Fixed response 404 + JSON body {"error":"endpoint not found"}
Listener: HTTP :80
Default: Redirect → HTTPS :443, StatusCode=HTTP_301
```

Target Group Health Check

```
healthCheck: {
path: "/actuator/health", // Spring Actuator
healthyThresholdCount: 2, // 2 consecutive successes = healthy
unhealthyThresholdCount: 3, // 3 consecutive failures = unhealthy
interval: Duration.seconds(30),
timeout: Duration.seconds(5),
healthyHttpCodes: "200",
}
deregistrationDelay: Duration.seconds(30), // drain fast for deployments
```

■ With deregistrationDelay=30s and spring.lifecycle.timeout-per-shutdown-phase=25s, ShopFlow achieves zero dropped requests during rolling deployments. ALB waits 30s for connections to drain before target is terminated.

04-05 **EBS Block Storage — Volume Types & Best Practices**

Elastic Block Store (EBS) provides durable block storage for EC2 instances. EBS volumes persist independently of the instance — they survive stop, restart, and even termination if delete-on-termination is disabled. ShopFlow encrypts all EBS volumes using KMS.

EBS Volume Type Selection Guide

Type	Max IOPS	Max Throughput	$/GB/month	Ideal Use Case
gp3 (General SSD)	16,000 IOPS (configurable)	1,000 MB/s (configurable)	$0.08	Default — OS, app, small DB. Always use over gp2.
gp2 (General SSD — legacy)	16,000 (3 IOPS/GB burst)	250 MB/s max	$0.10	Migrate to gp3. 20% more expensive.
io2 Block Express	256,000 IOPS	4,000 MB/s	$0.125	Oracle RAC, SQL Server on EC2, high-IOPS DB.
st1 (Throughput HDD)	500 IOPS	500 MB/s	$0.045	Kafka log storage, Hadoop sequential reads.
sc1 (Cold HDD)	250 IOPS	250 MB/s	$0.025	Archival backups, infrequent access, cheap storage.

gp3 vs gp2 — Why Always Choose gp3

- gp3 decouples IOPS and throughput from volume size — configure what you need
- gp3 baseline: 3,000 IOPS + 125 MB/s included in price regardless of volume size
- gp2 ties IOPS to size: 100GB gp2 = only 300 IOPS (below gp3 minimum)
- gp3 is 20% cheaper than gp2 for same storage size
- Additional gp3 IOPS: $0.005/provisioned IOPS above 3,000 (up to 16,000)
- Additional gp3 throughput: $0.04/MB/s above 125 MB/s (up to 1,000 MB/s)

Online Migration — gp2 to gp3 (Zero Downtime)

```
# Step 1: Modify volume type (live, no instance stop needed)
aws ec2 modify-volume \
--volume-id vol-0abc123def456 \
--volume-type gp3 \
--iops 3000 \
--throughput 125
# Step 2: Monitor modification progress
aws ec2 describe-volumes-modifications \
--volume-ids vol-0abc123def456 \
--query "VolumesModifications[].{State:ModificationState,Progress:Progress}"
# Transitions: modifying → optimizing → completed (typically 15-60 min)
# Step 3: Enable account-wide EBS encryption (all future volumes encrypted)
aws ec2 enable-ebs-encryption-by-default \
--region us-east-1
```

EBS Backup with Data Lifecycle Manager

```
# Create DLM policy for daily snapshots (retain 7 days)
aws dlm create-lifecycle-policy \
--description "ShopFlow daily EBS snapshots" \
--state ENABLED \
--execution-role-arn arn:aws:iam::123456789012:role/DLMRole \
--policy-details PolicyType=EBS_SNAPSHOT_MANAGEMENT,
ResourceTypes=[VOLUME],TargetTags=[{Key=Backup,Value=daily}],
Schedules=[{Name=DailyBackup,CreateRule:{Interval:24,
IntervalUnit:HOURS,Times:["03:00"]},
RetainRule:{Count:7}}]
```

■ Enable EBS encryption by default at the account level. All new EBS volumes and snapshots will be encrypted automatically with the AWS-managed KMS key. Zero performance overhead on Nitro-based instances (all current generation).

04-06 Spot Instances — 90% Cost Reduction

EC2 Spot Instances use spare AWS capacity at up to 90% discount. They can be reclaimed with 2 minutes notice. ShopFlow uses Spot for image thumbnail batch jobs and CI/CD runners — workloads that can checkpoint and resume.

Spot Suitability Matrix

Workload Type	Spot OK?	Rationale	ShopFlow Config
Image thumbnail generation	Yes — ideal	Stateless, checkpoint per image to S3	spot-batch-fleet (c6g.xlarge, c7g.xlarge)
CI/CD CodeBuild runners	Yes — ideal	Artifacts saved to S3/ECR before interrupt	CodeBuild spot capacity
ECS Fargate Spot	Yes — good	Fargate manages interruption gracefully	Non-prod ECS tasks only
ML model training	Yes — good	Checkpoint every 10min to S3	Planned for future ML workloads
SQS order consumers	No	In-flight messages lost on 2-min notice	On-Demand always
Customer-facing API (product-svc)	Never	Cannot gracefully drain in 2 minutes	On-Demand + Reserved
Database (RDS, Redis)	Never	Stateful — interruption = data loss risk	Reserved Instances always

Spot Interruption Handler — Spring Boot

```
@Scheduled(fixedDelay = 5000) // poll every 5 seconds
public void checkSpotInterruption() {
try {
ResponseEntity resp = restTemplate.getForEntity(
"http://169.254.169.254/latest/meta-data/spot/instance-action",
String.class);
if (resp.getStatusCode().is2xxSuccessful()) {
log.warn("SPOT INTERRUPTION — checkpointing batch progress");
batchService.checkpointCurrentJob(); // save state to SQS
sqsClient.sendMessage(dlqRequest); // return unprocessed items
context.close(); // graceful Spring shutdown
}
} catch (HttpClientErrorException e) {
// 404 = no interruption notice — continue working
}
}
```

■ Use priceCapacityOptimized allocation strategy. It selects instance pools with the highest available Spot capacity at the lowest price, significantly reducing interruption probability vs purely price-based (lowestPrice) selection.

04-07 EC2 Placement Groups

Placement Groups control the physical placement of EC2 instances to optimize for network performance (Cluster), fault isolation (Spread), or distributed-system rack-level redundancy (Partition).

Placement Group Types Compared

Type	Physical Placement	Network	Fault Tolerance	Max Instances	ShopFlow Use
Cluster	Same AZ, same rack	Sub-ms, 10/25Gbps	Low — rack failure = all down	No AWS limit	Image batch processing clus
Partition	Groups on separate racks	Normal inter-rack	High — rack-level isolation per partition	7 partitions per AZ	Future Kafka + ZooKeeper c
Spread	Each instance on separate rack	Normal inter-rack	Max — rack-level per instance	7 per AZ (strict limit)	OpenSearch masters, critica

CDK — Spread Group for OpenSearch Masters

```
const spreadPG = new ec2.PlacementGroup(this,"OpenSearchMasterSpread",{
strategy: ec2.PlacementGroupStrategy.SPREAD,
});
// Attach to ASG — each master node lands on separate physical rack
// If rack 1 fails, only 1 of 3 master nodes affected — cluster stays healthy
// Cluster group for batch processing (low-latency inter-node communication)
const clusterPG = new ec2.PlacementGroup(this,"BatchCluster",{
strategy: ec2.PlacementGroupStrategy.CLUSTER,
placementGroupName: "shopflow-image-batch",
});
```

Enhanced Networking — SR-IOV & ENA

Network Feature	Description	Performance	Availability
Enhanced Networking (SR-IOV)	Bypass hypervisor for NIC access	Up to 100Gbps, lower latency	All current-gen instances (automatic)
Elastic Network Adapter (ENA)	AWS custom NIC for high bandwidth	Up to 100Gbps, 800K pps	All Nitro instances (auto-enabled)
EFA (Elastic Fabric Adapter)	MPI-level interconnect for HPC	Sub-microsecond latency	p/hpc instance families only
Placement Group + ENA	Cluster PG + ENA combined	Best possible EC2-to-EC2 performance	For HPC, ML training clusters

■ Use Spread placement groups for ZooKeeper (3 nodes), OpenSearch masters (3 nodes), and Kafka controller nodes. Spread ensures no two critical nodes share the same physical rack, preventing correlated hardware failures.

04-08 Systems Manager — No-SSH Management

AWS Systems Manager (SSM) provides EC2 fleet management without SSH keys or open ports. ShopFlow uses SSM Session Manager for all production debugging sessions and Patch Manager for automated monthly OS patching.

SSM Session Manager vs Traditional SSH

Feature	Traditional SSH + Bastion	SSM Session Manager
Network port required	Port 22 open on bastion + all instances	Zero — HTTPS outbound (443) only via SSM
Authentication method	SSH private key file (theft/loss risk)	IAM permissions + optional MFA
Session audit trail	SSH logs (often partial, not centralized)	Full session recording in CloudTrail + S3
Browser-based access	No — requires local SSH client	Yes — AWS Console browser terminal
Cost	EC2 bastion: $20-50/month + key management	Free — SSM Agent included on Amazon Linux
Cross-region access	Requires VPN or jump chain	Any region from any client with IAM access
MFA requirement	Technically possible, rarely enforced	IAM Condition: aws:MultiFactorAuthPresent

SSM — Key Operations for ShopFlow

```
# Interactive shell on EC2 instance — no port 22 needed
aws ssm start-session --target i-0abc123 --profile shopflow-prod
# ECS Exec — shell inside running Fargate container (most common)
aws ecs execute-command \
--cluster shopflow-prod --task abc123def456 \
--container product-svc --command /bin/sh --interactive
# Run command across all product-svc instances simultaneously
aws ssm send-command \
--targets Key=tag:Service,Values=product-svc \
--document-name "AWS-RunShellScript" \
--parameters commands=["java -version","df -h","free -m","uptime"]
# Check SSM agent connectivity for all instances
aws ssm describe-instance-information \
--filters Key=tag:Environment,Values=prod \
--query "InstanceInformationList[].{ID:InstanceId,Ping:PingStatus,OS:PlatformName}"
```

■ Use Patch Manager with a Maintenance Window (Tuesday 2am, 2-hour window) for automated monthly patching. PCI-DSS requires critical patches applied within 30 days. SSM generates compliance reports for auditors automatically.

04-09 CloudWatch Agent — Custom Metrics & Log Shipping

Default EC2 CloudWatch metrics include CPU utilization, network I/O, and disk I/O — but NOT memory utilization, disk space, or per-process metrics. The CloudWatch Agent fills this gap and ships application logs to CloudWatch Logs for centralized analysis via Log Insights.

CloudWatch Agent Configuration

```
{
"agent": {"metrics_collection_interval": 60, "run_as_user": "cwagent"},
"metrics": {
"namespace": "ShopFlow/EC2",
"metrics_collected": {
"mem": {"measurement": ["mem_used_percent","mem_available_percent"]},
"disk": {"measurement": ["disk_used_percent"],
"resources": ["/","/var/log","/tmp"]},
"cpu": {"totalcpu": true,
"measurement": ["cpu_usage_user","cpu_usage_system","cpu_usage_idle"]}
},
"append_dimensions": {
"AutoScalingGroupName": "${aws:AutoScalingGroupName}",
"InstanceId": "${aws:InstanceId}"
}
},
"logs": {
"logs_collected": {"files": {"collect_list": [
{"file_path": "/var/log/shopflow/product-svc/*.log",
"log_group_name": "/ec2/shopflow/product-svc",
"log_stream_name": "{instance_id}",
"timestamp_format": "%Y-%m-%d %H:%M:%S"},
{"file_path": "/var/log/shopflow/gc.log",
"log_group_name": "/ec2/shopflow/gc-logs",
"log_stream_name": "{instance_id}"}
]}}
}
```

Scale on Memory — CDK Step Scaling

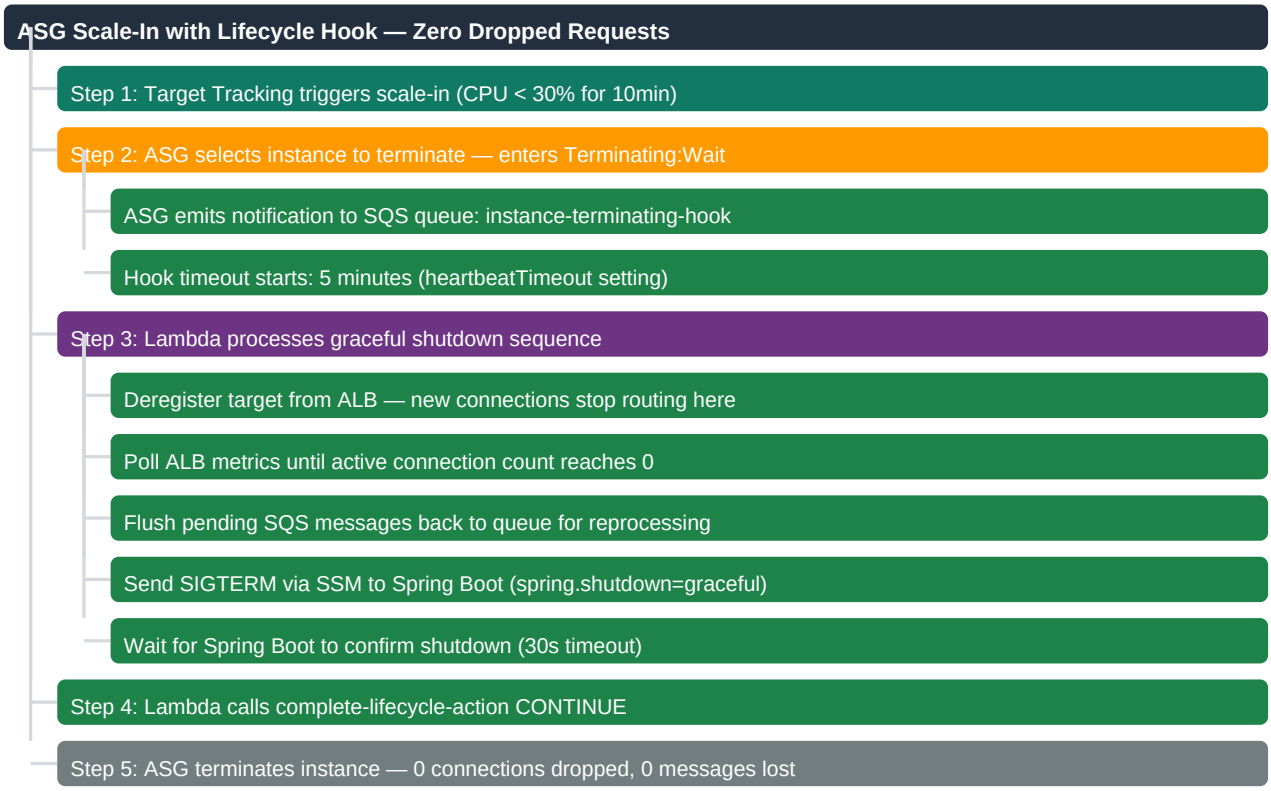
```
asg.scaleOnMetric("MemoryScaling", {
metric: new cloudwatch.Metric({
namespace: "ShopFlow/EC2",
metricName: "mem_used_percent",
dimensionsMap: {AutoScalingGroupName: asg.autoScalingGroupName},
statistic: "Average",
period: Duration.minutes(1),
}),
scalingSteps: [
{upper: 50, change: -1}, // mem < 50%: remove 1 instance
{lower: 75, change: +2}, // mem > 75%: add 2 instances
{lower: 90, change: +4}, // mem > 90%: emergency add 4
],
adjustmentType: autoscaling.AdjustmentType.CHANGE_IN_CAPACITY,
cooldown: Duration.seconds(120),
});
```

■ Enable EC2 Detailed Monitoring ($0.014/metric/month) for 1-minute granularity. Default 5-minute metrics cause ASG to react 15 minutes after a spike starts. With 1-minute metrics, ASG adds capacity in 3-5 minutes.

04-10 Lifecycle Hooks — Zero Drop-Requests on Scale Events

Lifecycle Hooks pause ASG actions during instance launch or termination to allow custom processing. ShopFlow uses a termination hook to drain ALB connections and flush in-flight SQS messages before the instance is terminated.

Lifecycle State Machine



Lifecycle Hook CDK Configuration

```
asg.addLifecycleHook("GracefulTerminate", {
lifecycleTransition: autoscaling.LifecycleTransition.INSTANCE_TERMINATING,
heartbeatTimeout: Duration.minutes(5), // Lambda must respond in 5min
notificationTarget: new hooks.QueueHook(lifecycleQueue),
defaultResult: autoscaling.DefaultResult.CONTINUE, // terminate if Lambda fails
});
// Lambda: complete lifecycle action after graceful shutdown
await asg.completeLifecycleAction({
LifecycleHookName: "GracefulTerminate",
AutoScalingGroupName: "shopflow-product-asg",
InstanceId: instanceId,
LifecycleActionResult: "CONTINUE" // or ABANDON to cancel termination
});
```

■ Spring Boot config for graceful shutdown: server.shutdown=graceful, spring.lifecycle.timeout-per-shutdown-phase=25s. This ensures all in-flight HTTP requests complete before the application exits after receiving SIGTERM.

04-11 EC2 Cost Optimization — Full Strategy

EC2 and compute typically account for 35-50% of a startup's AWS bill. ShopFlow reduced its compute costs by 84% through a combination of rightsizing, commitment discounts, and architectural changes.

ShopFlow EC2 Before vs After Optimization

Component	Before ($/month)	After ($/month)	% Saved	Strategy Applied
Web tier (6x m5.xlarge OD)	$828	$118 (4x m6g.l Savings Plan)	86%	Rightsize + Savings Plan
Dev environments (4 instances 24/7)	$276	$69 (auto-stop nights/wkends)	75%	Instance Scheduler
Image batch (On-Demand)	$200	$20 (Spot Fleet c6g.xlarge)	90%	Spot Instances
CI/CD CodeBuild workers	$120	$24 (Spot capacity)	80%	Spot CodeBuild
Total EC2 monthly cost	$1,424	$231	84%	All strategies combined

■ Run AWS Compute Optimizer every month. It identifies oversized instances by analyzing 14 days of CloudWatch CPU/memory data. ShopFlow found 4 instances running at <10% average CPU — all downsized with zero performance impact.

04-12 Black Friday Scaling Runbook

Black Friday brings 10x normal traffic between 6pm-11pm EST. Pre-scaling is critical — Auto Scaling reacts to traffic after the fact (3-5 minute lag), but Scheduled + Predictive Scaling ensures capacity is ready before customers arrive.

Pre-Event Checklist (T-24h to T-0)

Time	Action	Tool/Command	Expected Outcome
T-24h	Increase all ASG desired to 10	set-desired-capacity	Warm instances ready overnight
T-24h	Disable Spot for event (100% OD)	Update mixedInstancesPolicy	No interruption risk during peak
T-2h	Pre-warm Redis cache (top 1,000 SKUs)	Invoke warm-cache Lambda	Cache hit rate >90% at event start
T-1h	Scale order-svc ASG to 8 instances	set-desired-capacity on order-asg	8 checkout processors ready
T-1h	Increase maxCapacity to 40 on all ASGs	Update ASG max via CLI	Headroom for 4x further growth
T-30m	Verify all ALB target health checks green	describe-target-health	All targets: InService
T-15m	Enable 1-minute CloudWatch detailed metrics	Already enabled in prod	Fast scaling decisions
T-0	Event live — watch CloudWatch dashboard	All hands on standby mode	Observe, react if needed
T+5h	Restore normal ASG settings (min=2, max=20, desired=4)	Scheduled action	Return to cost-efficient steady state

```
# Scheduled scale-up at 5:30pm ET (22:30 UTC) — Black Friday
aws autoscaling put-scheduled-update-group-action \
--auto-scaling-group-name shopflow-product-asg \
--scheduled-action-name BF2024ScaleUp \
--recurrence "30 22 29 11 5" \ # 5:30pm ET, Nov 29, Friday
--min-size 10 --max-size 40 --desired-capacity 15
```

■ Use Predictive Scaling after 14+ days of history — it pre-scales 10 minutes before the predicted spike based on ML pattern matching, much faster than Target Tracking which reacts after the spike begins.

04-13 Reserved Instances & Savings Plans

Commitment discounts deliver the highest cost savings for stable workloads. ShopFlow commits on its minimum always-on capacity, saving $563/month vs pure On-Demand pricing.

Savings Plans vs Reserved Instances — Decision Guide

Dimension	Compute Savings Plan	EC2 Savings Plan	Reserved Instance
Applies To	EC2 + Fargate + Lambda (any)	EC2 only (specific instance family)	Specific instance type + AZ/Region
Maximum Discount	66% vs On-Demand	72% vs On-Demand	72% (3yr All Upfront)
Flexibility	Any EC2 type, size, region, OS	Same family, any size, same region	Fixed type, size, tenancy
Payment options	All Upfront, Partial, No Upfront	All, Partial, No Upfront	All, Partial, No Upfront
Minimum Term	1 year	1 year	1 year
ShopFlow Use	Fargate tasks + Lambda baseline	Not used (prefer Fargate SP)	RDS Aurora + ElastiCache

ShopFlow Monthly Savings Breakdown

Commitment Purchase	Term	On-Demand Rate	Committed Rate	Monthly Saving
Compute SP (8 Fargate vCPU baseline)	1yr No Upfront	$0.048/vCPU-hr	$0.023/vCPU-hr	$78
EC2 SP (4x c6g.xlarge web tier)	1yr No Upfront	$0.136/hr	$0.054/hr	$198
RDS Aurora db.r6g.large Reserved	1yr All Upfront	$0.277/hr	$0.154/hr	$180
ElastiCache cache.r6g.large x2 RI	1yr All Upfront	$0.163/hr each	$0.090/hr each	$107
Total Commitment Savings				$563/month = $6,756/year

Monitoring Commitment Utilization

```
# Savings Plan coverage (target: 70-80%)
aws ce get-savings-plans-coverage \
--time-period Start=2024-10-01,End=2024-11-01 \
--granularity MONTHLY
# Savings Plan utilization (target: >90%)
aws ce get-savings-plans-utilization \
--time-period Start=2024-10-01,End=2024-11-01 \
--granularity MONTHLY
# < 80% utilization = bought too much, paying for unused commitment
# > 95% utilization = may want to buy more to cover growing baseline
```

■ Start with 1-Year No Upfront Savings Plans — lowest lock-in, still 46% savings. After 3 months of stable usage, upgrade some to All Upfront 3-Year for maximum 72% savings on proven stable workloads.

04-14 Observability — Metrics, Alarms & Log Insights

ShopFlow's production EC2 fleet requires three observability layers: infrastructure metrics for scaling decisions, application logs for debugging, and actionable alarms for on-call response.

Key Alarm Thresholds — Production

Metric	Source	Warning	Critical	On-Call Action
CPUUtilization	EC2 native	70% 10min	85% 5min	Scale out +2, investigate at critical
mem_used_percent	CW Agent	75% 5min	90% 2min	Scale out +3, check for memory leak
disk_used_percent	CW Agent	80% 15min	90% 5min	Alert ops: expand EBS or cleanup logs

Metric	Source	Warning	Critical	On-Call Action
ALB p99 ResponseTime	ALB native	500ms 5min	2s 2min	PagerDuty P1 — checkout flow impacted
ALB 5xxCount	ALB native	>50/min	> 200/min	PagerDuty P1 — service errors
StatusCheckFailed_System	EC2 native	Any failure	2 failures/5min	EC2 Auto Recovery or ASG replacement

Log Insights — ShopFlow Query Library

```
# Query 1: Error rate by service in last 1 hour
fields @timestamp, @logStream, @message
| filter @message like /ERROR|EXCEPTION/
| stats count(*) as errors by @logStream
| sort errors desc | limit 20
# Query 2: Slow API requests (P95 candidates)
fields @timestamp, duration, path, userId
| filter @logStream like /product-svc/
| filter ispresent(duration)
| stats avg(duration), pct(duration,95), count() by path
| sort avg_duration desc | limit 10
# Query 3: Memory trend per ASG (detect gradual leak)
fields @timestamp, mem_used_percent, InstanceId
| filter MetricName = "mem_used_percent"
| stats avg(mem_used_percent) by bin(30min), AutoScalingGroupName
```

■ Create a CloudWatch Dashboard for each service: ASG capacity, CPU/memory, ALB error rates, RDS connection count, SQS queue depth. Bookmark it in PagerDuty runbooks so on-call engineers access it in seconds.

04-BP Best Practices & Anti-Patterns

✓ Use Graviton (ARM) instances for Java workloads — 40% better price/performance

✓ Always use gp3 over gp2 — independent IOPS/throughput, 20% cheaper

✓ Enforce IMDSv2 (requireImdsv2: true) to block SSRF credential theft attacks

✓ Set ASG health check type to ELB — catches Spring Boot failures not just EC2 hardware

✓ Use Launch Templates (Launch Configurations are deprecated and featureless)

✓ Enable EC2 Detailed Monitoring for 1-minute granularity on production ASGs

✓ Add Lifecycle Hooks for graceful termination — zero dropped requests on scale-in

✓ Use Spot Instances for batch and CI/CD — priceCapacityOptimized + diverse types

✓ Buy Compute Savings Plans for stable Fargate/Lambda baseline — 66% savings

■ Never place stateful data on EC2 instance store — lost on stop, terminate, hardware failure

■ Do not set ASG cooldown to 0 seconds — causes rapid scale up/down thrashing

■ Do not use default VPC for production — lacks proper subnet and SG isolation

■ Do not run Spot Instances for SQS consumers, stateful apps, or customer-facing services

■ Never attach AdministratorAccess IAM policy to EC2 instances — use least-privilege roles

04-QR Quick Reference & Interview Q&A

Concept	Key Fact
EC2 billing unit	Per second, 60-second minimum for On-Demand and Reserved
Spot interruption warning	2 minutes — poll IMDS /spot/instance-action every 5s
Spot max discount	Up to 90% vs On-Demand with priceCapacityOptimized
gp3 baseline specs	3,000 IOPS + 125 MB/s included in base price ($0.08/GB/month)
ASG default cooldown	300 seconds — prevents thrashing between scale-up and scale-down
ALB deregistration delay	Default 300s — set 30s for fast deploys (matches Spring graceful)
IMDSv2 benefit	Requires PUT token — blocks SSRF attacks stealing EC2 IAM credentials
Placement Spread limit	7 instances per Availability Zone maximum
Graviton performance gain	35-50% better TPS for Java Spring Boot vs same-class x86 instance
EBS max volume size	64 TiB per volume (io2 Block Express)
ASG health check grace	90s for Spring Boot — JVM startup + Spring context load + health check
Compute Savings Plan discount	66% vs On-Demand (1-year, No Upfront Savings Plan)
EC2 Savings Plan discount	72% vs On-Demand (3-year, All Upfront, specific instance family)

Top Interview Questions

Question	Answer
How does ASG handle instance failure?	Health check (ELB or EC2) marks instance unhealthy → ASG terminates and launches replace
ALB vs NLB — when to use each?	ALB: Layer 7, path/host/header routing, WebSocket, gRPC, Lambda targets. Best for REST AP
gp3 vs gp2?	gp3: 3000 IOPS + 125MB/s base, configure IOPS/throughput independently of size, 20% cheap
IMDSv2 — why enforce it?	Requires PUT request for session token before any metadata GET requests. Blocks SSRF atta
Zero-downtime AMI update with ASG?	Instance Refresh: rolling replace with MinHealthyPercentage=90%. ALB drains connections (de
Spot vs Reserved vs On-Demand?	Spot: 90% off, fault-tolerant batch only, 2-min interrupt notice. Reserved/Savings Plan: 40-72%
What is a Lifecycle Hook?	Pauses ASG before launching or terminating instances. On terminate: drain ALB, flush SQS bu